

2.1. Intro to CUDA C++

This chapter introduces some of the basic concepts of the CUDA programming model by illustrating how they are exposed in C++.

This programming guide focuses on the CUDA runtime API. The CUDA runtime API is the most commonly used way of using CUDA in C++ and is built on top of the lower level CUDA driver API.

[CUDA Runtime API and CUDA Driver API](#) discusses the difference between the APIs and [CUDA driver API](#) discusses writing code that mixes the APIs.

This guide assumes the CUDA Toolkit and NVIDIA Driver are installed and that a supported NVIDIA GPU is present. See [The CUDA Quickstart Guide](#) for instructions on installing the necessary CUDA components.

2.1.1. Compilation with NVCC

GPU code written in C++ is compiled using the NVIDIA Cuda Compiler, `nvcc`. `nvcc` is a compiler driver that simplifies the process of compiling C++ or PTX code: It provides simple and familiar command line options and executes them by invoking the collection of tools that implement the different compilation stages.

This guide will show `nvcc` command lines which can be used on any Linux system with the CUDA Toolkit installed, at a Windows command line or power shell, or on Windows Subsystem for Linux with the CUDA Toolkit. The [nvcc chapter](#) of this guide covers common use cases of `nvcc`, and complete documentation is provided by [the nvcc user manual](#).

2.1.2. Kernels

As mentioned in the introduction to the [CUDA Programming Model](#), functions which execute on the GPU which can be invoked from the host are called kernels. Kernels are written to be run by many parallel threads simultaneously.

2.1.2.1. Specifying Kernels

The code for a kernel is specified using the `__global__` declaration specifier. This indicates to the compiler that this function will be compiled for the GPU in a way that allows it to be invoked from a kernel launch. A kernel launch is an operation which starts a kernel running, usually from the CPU. Kernels are functions with a `void` return type.

```
// Kernel definition
__global__ void vecAdd(float* A, float* B, float* C)
{
}
```

2.1.2.2. Launching Kernels

The number of threads that will execute the kernel in parallel is specified as part of the kernel launch. This is called the execution configuration. Different invocations of the same kernel may use different execution configurations, such as a different number of threads or thread blocks.

There are two ways of launching kernels from CPU code, [triple chevron notation](#) and `cudaLaunchKernelEx`. Triple chevron notation, the most common way of launching kernels, is introduced here. An example of launching a kernel using `cudaLaunchKernelEx` is shown and discussed in detail in [Section 3.1.1](#).

2.1.2.2.1. Triple Chevron Notation

Triple chevron notation is a [CUDA C++ Language Extension](#) which is used to launch kernels. It is called triple chevron because it **uses three chevron characters to encapsulate the execution configuration** for the kernel launch, i.e. `<<< >>>`. Execution configuration parameters are specified as a comma separated list inside the chevrons, similar to parameters to a function call. The syntax for a kernel launch of the `vecAdd` kernel is shown below.

```
__global__ void vecAdd(float* A, float* B, float* C)
{
}

int main()
{
    ...
    // Kernel invocation
    vecAdd<<<1, 256>>>(A, B, C);
    ...
}
```

The first two parameters to the triple chevron notation are the **grid dimensions and the thread block dimensions**, respectively. When using 1-dimensional thread blocks or grids, integers can be used to specify dimensions.

The above code launches a single thread block containing 256 threads. Each thread will execute the exact same kernel code. In [Thread and Grid Index Intrinsic](#)s, we'll show how each thread can use its index within the thread block and grid to change the data it operates on.

There is a limit to the number of threads per block, since all threads of a block reside on the same streaming multiprocessor(SM) and must share the resources of the SM. On current GPUs, a thread block may contain up to 1024 threads. If resources allow, more than one thread block can be scheduled on an SM simultaneously.

Kernel launches are asynchronous with respect to the host thread. That is, the kernel will be setup for execution on the GPU, but the host code will not wait for the kernel to complete (or even start) executing on the GPU before proceeding. Some form of synchronization between the GPU and CPU must be used to determine that the kernel has completed. The most basic version, completely synchronizing the entire GPU, is shown in [Synchronizing CPU and GPU](#). More sophisticated methods of synchronization are covered in [Asynchronous Execution](#).

When using **2 or 3-dimensional grids or thread blocks**, the CUDA **type `dim3` is used** as the grid and thread block dimension parameters. The code fragment below shows a kernel launch of a `MatAdd` kernel using 16 by 16 grid of thread blocks, each thread block is 8 by 8.

```
int main()
{
    ...
    dim3 grid(16,16);
    dim3 block(8,8);
    MatAdd<<<grid, block>>>(A, B, C);
    ...
}
```

2.1.2.3. Thread and Grid Index Intrinsic

Within kernel code, CUDA **provides intrinsics to access** parameters of the execution configuration and the **index of a thread or block**.

- `threadIdx` gives the index of a thread within its thread block. Each thread in a thread block will have a different index.

- `blockDim` gives the dimensions of the thread block, which was specified in the execution configuration of the kernel launch.
- `blockIdx` gives the index of a thread block within the grid. Each thread block will have a different index.
- `gridDim` gives the dimensions of the grid, which was specified in the execution configuration when the kernel was launched.

Each of these intrinsics is a 3-component vector with a `.x`, `.y`, and `.z` member. Dimensions not specified by a launch configuration will default to 1. `threadIdx` and `blockIdx` are zero indexed. That is, `threadIdx.x` will take on values from 0 up to and including `blockDim.x-1`. `.y` and `.z` operate the same in their respective dimensions.

Similarly, `blockIdx.x` will have values from 0 up to and including `gridDim.x-1`, and the same for `.y` and `.z` dimensions, respectively.

These allow an individual thread to identify what work it should carry out. Returning to the `vecAdd` kernel, the kernel takes three parameters, each is a vector of floats. The kernel performs an element-wise addition of `A` and `B` and stores the result in `C`. The kernel is parallelized such that each thread will perform one addition. Which element it computes is determined by its thread and grid index.

```
__global__ void vecAdd(float* A, float* B, float* C)
{
    // calculate which element this thread is responsible for computing
    int workIndex = threadIdx.x + blockDim.x * blockIdx.x

    // Perform computation
    C[workIndex] = A[workIndex] + B[workIndex];
}

int main()
{
    ...
    // A, B, and C are vectors of 1024 elements
    vecAdd<<<4, 256>>>(A, B, C);
    ...
}
```

In this example, 4 thread blocks of 256 threads are used to add a vector of 1024 elements. In the first thread block, `blockIdx.x` will be zero, and so each thread's `workIndex` will simply be its `threadIdx.x`. In the second thread block, `blockIdx.x` will be 1, so `blockDim.x * blockIdx.x` will be the same as `blockDim.x`, which is 256 in this case. The `workIndex` for each thread in the second thread block will be its `threadIdx.x + 256`. In the third thread block `workIndex` will be `threadIdx.x + 512`.

This computation of `workIndex` is very common for 1-dimensional parallelizations. Expanding to two or three dimensions often follows the same pattern in each of those dimensions.

2.1.2.3.1. Bounds Checking

The example given above assumes that the length of the vector is a multiple of the thread block size, 256 threads in this case. To make the kernel handle any vector length, we can add checks that the memory access is not exceeding the bounds of the arrays as shown below, and then launch one thread block which will have some inactive threads.

```
__global__ void vecAdd(float* A, float* B, float* C, int vectorLength)
{
    // calculate which element this thread is responsible for computing
    int workIndex = threadIdx.x + blockDim.x * blockIdx.x

    if(workIndex < vectorLength)
    {
        // Perform computation
        C[workIndex] = A[workIndex] + B[workIndex];
    }
}
```

With the above kernel code, more threads than needed can be launched without causing out-of-bounds accesses to the arrays. When `workIndex` exceeds `vectorLength`, threads exit and do not do any work. Launching extra threads in a block that do no work does not incur a large overhead cost, however launching thread blocks in which no threads do work should be avoided. This kernel can now handle vector lengths which are not a multiple of the block size.

The number of thread blocks which are needed can be calculated as the ceiling of the number of threads needed, the vector length in this case, divided by the number of threads per block. That is, the integer division of the number of threads needed by the number of threads per block, rounded up. A common way of expressing this as a single integer division is given below. By adding `threads - 1` before the integer division, this behaves like a ceiling function, adding another thread block only if the vector length is not divisible by the number of threads per block.

```
// vectorLength is an integer storing number of elements in the vector
int threads = 256;
int blocks = (vectorLength + threads-1)/threads;
vecAdd<<<blocks, threads>>>(devA, devB, devC, vectorLength);
```

The [CUDA Core Compute Library \(CCCL\)](#) provides a convenient utility, `cuda::ceil_div`, for doing this ceiling divide to calculate the number of blocks needed for a kernel launch. This utility is available by including the header `<cuda/cmath>`.

```
// vectorLength is an integer storing number of elements in the vector
int threads = 256;
int blocks = cuda::ceil_div(vectorLength, threads);
vecAdd<<<blocks, threads>>>(devA, devB, devC, vectorLength);
```

The choice of 256 threads per block here is arbitrary, but this is quite often a good value to start with.

2.1.3. Memory in GPU Computing

In order to use the `vecAdd` kernel shown above, the arrays `A`, `B`, and `C` must be in memory accessible to the GPU. There are several different ways to do this, two of which will be illustrated here. Other methods will be covered in later sections on [unified memory](#). The memory spaces available to code running on the GPU were introduced in [GPU Memory](#) and are covered in more detail in [GPU Device Memory Spaces](#).

2.1.3.1. Unified Memory

Unified memory is a feature of the CUDA runtime which lets the NVIDIA Driver manage movement of data between host and device(s). Memory is allocated using the `cudaMallocManaged` API or by declaring a variable with the `__managed__` specifier. The NVIDIA Driver will make sure that the memory is accessible to the GPU or CPU whenever either tries to access it.

The code below shows a complete function to launch the `vecAdd` kernel which uses unified memory for the input and output vectors that will be used on the GPU. `cudaMallocManaged` allocates buffers which can be accessed from either the CPU or the GPU. These buffers are released using `cudaFree`.


```

void unifiedMemExample(int vectorLength)
{
    // Pointers to memory vectors
    float* A = nullptr;
    float* B = nullptr;
    float* C = nullptr;
    float* comparisonResult = (float*)malloc(vectorLength*sizeof(float));

    // Use unified memory to allocate buffers
    cudaMallocManaged(&A, vectorLength*sizeof(float));
    cudaMallocManaged(&B, vectorLength*sizeof(float));
    cudaMallocManaged(&C, vectorLength*sizeof(float));

    // Initialize vectors on the host
    initArray(A, vectorLength);
    initArray(B, vectorLength);

    // Launch the kernel. Unified memory will make sure A, B, and C are
    // accessible to the GPU
    int threads = 256;
    int blocks = cuda::ceil_div(vectorLength, threads);
    vecAdd<<<blocks, threads>>>(A, B, C, vectorLength);
    // Wait for the kernel to complete execution
    cudaDeviceSynchronize();

    // Perform computation serially on CPU for comparison
    serialVecAdd(A, B, comparisonResult, vectorLength);

    // Confirm that CPU and GPU got the same answer
    if(vectorApproximatelyEqual(C, comparisonResult, vectorLength))
    {
        printf("Unified Memory: CPU and GPU answers match\n");
    }
    else
    {
        printf("Unified Memory: Error - CPU and GPU answers do not match\n");
    }

    // Clean Up
    cudaFree(A);
    cudaFree(B);
    cudaFree(C);
    free(comparisonResult);
}

```

Unified memory is supported on all operating systems and GPUs supported by CUDA, though the underlying mechanism and performance may differ based on system architecture. [Unified Memory](#) provides more details. On some Linux systems, (e.g. those with [address translation services](#) or [heterogeneous memory management](#)) all system memory is automatically unified memory, and there is no need to use `cudaMallocManaged` or the `__managed__` specifier.

2.1.3.2. Explicit Memory Management

Explicitly managing memory allocation and data migration between memory spaces can [help improve application performance](#), though it does make for [more verbose code](#). The code below explicitly allocates memory on the GPU using `cudaMalloc`. Memory on the GPU is freed using the same `cudaFree` API as was used for unified memory in the previous example.

```

void explicitMemExample(int vectorLength)
{
    // Pointers for host memory
    float* A = nullptr;
    float* B = nullptr;
    float* C = nullptr;
    float* comparisonResult = (float*)malloc(vectorLength*sizeof(float));

    // Pointers for device memory
    float* devA = nullptr;
    float* devB = nullptr;
    float* devC = nullptr;

    //Allocate Host Memory using cudaMallocHost API. This is best practice
    // when buffers will be used for copies between CPU and GPU memory
    cudaMallocHost(&A, vectorLength*sizeof(float));
    cudaMallocHost(&B, vectorLength*sizeof(float));
    cudaMallocHost(&C, vectorLength*sizeof(float));

    // Initialize vectors on the host
    initArray(A, vectorLength);
    initArray(B, vectorLength);

    // start-allocate-and-copy
    // Allocate memory on the GPU
    cudaMalloc(&devA, vectorLength*sizeof(float));
    cudaMalloc(&devB, vectorLength*sizeof(float));
    cudaMalloc(&devC, vectorLength*sizeof(float));

    // Copy data to the GPU
    cudaMemcpy(devA, A, vectorLength*sizeof(float), cudaMemcpyDefault);
    cudaMemcpy(devB, B, vectorLength*sizeof(float), cudaMemcpyDefault);
    cudaMemcpy(devC, 0, vectorLength*sizeof(float));
    // end-allocate-and-copy

    // Launch the kernel
    int threads = 256;
    int blocks = cuda::ceil_div(vectorLength, threads);
    vecAdd<<<blocks, threads>>>(devA, devB, devC, vectorLength);
    // wait for kernel execution to complete
    cudaDeviceSynchronize();

    // Copy results back to host
    cudaMemcpy(C, devC, vectorLength*sizeof(float), cudaMemcpyDefault);

    // Perform computation serially on CPU for comparison
    serialVecAdd(A, B, comparisonResult, vectorLength);

    // Confirm that CPU and GPU got the same answer
    if(vectorApproximatelyEqual(C, comparisonResult, vectorLength))
    {
        printf("Explicit Memory: CPU and GPU answers match\n");
    }
    else
    {
        printf("Explicit Memory: Error - CPU and GPU answers to not match\n");
    }

    // clean up
    cudaFree(devA);
    cudaFree(devB);
    cudaFree(devC);
    cudaFreeHost(A);
    cudaFreeHost(B);
    cudaFreeHost(C);
    free(comparisonResult);
}

```

The CUDA API `cudaMemcpy` is used to copy data from a buffer residing on the CPU to a buffer residing on the GPU. Along with the destination pointer, source pointer, and size in bytes, the final parameter of `cudaMemcpy` is a `cudaMemcpyKind_t`. This can have values such as `cudaMemcpyHostToDevice` for copies from the CPU to a GPU, `cudaMemcpyDeviceToHost` for copies from the GPU to the CPU, or `cudaMemcpyDeviceToDevice` for copies within a GPU or between GPUs.

In this example, `cudaMemcpyDefault` is passed as the last argument to `cudaMemcpy`. This causes CUDA to use the value of the source and destination pointers to determine the type of copy to perform.

The `cudaMemcpy` API is synchronous. That is, it does not return until the copy has completed. Asynchronous copies are introduced in [Launching Memory Transfers in CUDA Streams](#).

The code uses `cudaMallocHost` to allocate memory on the CPU. This allocates page-locked memory on the host, which can improve copy performance and is necessary for asynchronous memory transfers. In general, it is good practice to use page-locked memory for CPU buffers that will be used in data transfers to and from GPUs. Performance can degrade on some systems if too much host memory is page-locked. Best practice is to page-lock only buffers which will be used for sending or receiving data from the GPU.

2.1.3.3. Memory Management and Application Performance

As can be seen in the above example, explicit memory management is more verbose, requiring the programmer to specify copies between the host and device. This is the advantage and disadvantage of explicit memory management: it affords more control of when data is copied between host and devices, where memory is resident, and exactly what memory is allocated where. Explicit memory management can provide performance opportunities controlling memory transfers and overlapping them with other computations.

When using unified memory, there are CUDA APIs (which will be covered in [Memory Advise and Prefetch](#)), which provide hints to the NVIDIA driver managing the memory, which can enable some of the performance benefits of using explicit memory management when using unified memory.

2.1.4. Synchronizing CPU and GPU

As mentioned in [Launching Kernels](#), kernel launches are asynchronous with respect to the CPU thread which called them. This means the control flow of the CPU thread will continue executing before the kernel has completed, and possibly even before it has launched. In order to guarantee that a kernel has completed execution before proceeding in host code, some synchronization mechanism is necessary.

The simplest way to synchronize the GPU and a host thread is with the use of `cudaDeviceSynchronize`, which blocks the host thread until all previously issued work on the GPU has completed. In the examples of this chapter this is sufficient because only single operations are being executed on the GPU. In larger applications, there may be multiple streams executing work on the GPU and `cudaDeviceSynchronize` will wait for work in all streams to complete. In these applications, using [Stream Synchronization](#) APIs to synchronize only with a specific stream or [CUDA Events](#) is recommended. These will be covered in detail in the [Asynchronous Execution](#) chapter.

2.1.5. Putting it All Together

The following listings show the entire code for the simple vector addition kernel introduced in this chapter along with all host code and utility functions for checking to verify that the answer obtained is correct. These examples default to using a vector length of 1024, but accept a different vector length as a command line argument to the executable.

Unified Memory

Explicit Memory Management

```
#include <cuda_runtime_api.h>
#include <memory.h>
#include <cstdlib>
#include <ctime>
#include <stdio.h>
#include <cuda/cmath>

__global__ void vecAdd(float* A, float* B, float* C, int vectorLength)
{
    int workIndex = threadIdx.x + blockIdx.x*blockDim.x;
    if(workIndex < vectorLength)
    {
        C[workIndex] = A[workIndex] + B[workIndex];
    }
}

void initArray(float* A, int length)
{
    std::srand(std::time({}));
    for(int i=0; i<length; i++)
    {
        A[i] = rand() / (float)RAND_MAX;
    }
}

void serialVecAdd(float* A, float* B, float* C, int length)
{
    for(int i=0; i<length; i++)
    {
        C[i] = A[i] + B[i];
    }
}

bool vectorApproximatelyEqual(float* A, float* B, int length, float epsilon=0.00001)
{
    for(int i=0; i<length; i++)
    {
        if(fabs(A[i] - B[i]) > epsilon)
        {
            printf("Index %d mismatch: %f != %f", i, A[i], B[i]);
            return false;
        }
    }
    return true;
}

//unified-memory-begin
void unifiedMemExample(int vectorLength)
{
    // Pointers to memory vectors
    float* A = nullptr;
    float* B = nullptr;
    float* C = nullptr;
    float* comparisonResult = (float*)malloc(vectorLength*sizeof(float));

    // Use unified memory to allocate buffers
    cudaMallocManaged(&A, vectorLength*sizeof(float));
    cudaMallocManaged(&B, vectorLength*sizeof(float));
    cudaMallocManaged(&C, vectorLength*sizeof(float));

    // Initialize vectors on the host
    initArray(A, vectorLength);
    initArray(B, vectorLength);

    // Launch the kernel. Unified memory will make sure A, B, and C are
    // accessible to the GPU
    int threads = 256;
    int blocks = cuda::ceil_div(vectorLength, threads);
    vecAdd<<<blocks, threads>>>(A, B, C, vectorLength);
    // Wait for the kernel to complete execution
    cudaDeviceSynchronize();

    // Perform computation serially on CPU for comparison
    serialVecAdd(A, B, comparisonResult, vectorLength);

    // Confirm that CPU and GPU got the same answer
```



```

    if(vectorApproximatelyEqual(C, comparisonResult, vectorLength))
    {
        printf("Unified Memory: CPU and GPU answers match\n");
    }
    else
    {
        printf("Unified Memory: Error - CPU and GPU answers do not match\n");
    }

    // Clean Up
    cudaFree(A);
    cudaFree(B);
    cudaFree(C);
    free(comparisonResult);
}
//unified-memory-end

int main(int argc, char** argv)
{
    int vectorLength = 1024;
    if(argc >=2)
    {
        vectorLength = std::atoi(argv[1]);
    }
    unifiedMemExample(vectorLength);
    return 0;
}

```

These can be built and run using *nvcc* as follows:

```

$ nvcc vecAdd_unifiedMemory.cu -o vecAdd_unifiedMemory
$ ./vecAdd_unifiedMemory
Unified Memory: CPU and GPU answers match
$ ./vecAdd_unifiedMemory 4096
Unified Memory: CPU and GPU answers match

```

```

$ nvcc vecAdd_explicitMemory.cu -o vecAdd_explicitMemory
$ ./vecAdd_explicitMemory
Explicit Memory: CPU and GPU answers match
$ ./vecAdd_explicitMemory 4096
Explicit Memory: CPU and GPU answers match

```

In these examples, all threads are doing independent work and do not need to coordinate or synchronize with each other. Frequently, threads will need to cooperate and communicate with other threads to carry out their work. Threads within a block can share data through shared memory and synchronize to coordinate memory accesses.

The most basic mechanism for synchronization at the block level is the `__syncthreads()` intrinsic, which acts as a barrier at which all threads in the block must wait before any threads are allowed to proceed. Shared Memory gives an example of using shared memory.

For efficient cooperation, shared memory is expected to be a low-latency memory near each processor core (much like an L1 cache) and `__syncthreads()` is expected to be lightweight. `__syncthreads()` only synchronizes the threads within a single thread block. Synchronization between blocks is not supported by the CUDA programming model. Cooperative Groups provides mechanism to set synchronization domains other than a single thread block.

Best performance is usually achieved when synchronization is kept within a thread block. Thread blocks can still work on common results using atomic memory functions, which will be covered in coming sections.

Section Section 3.2.4 covers CUDA synchronization primitives that provide very fine-grained control for maximizing performance and resource utilization.

2.1.6. Runtime Initialization

The `CUDA runtime` creates a `CUDA context` for each device in the system. This context is the primary context for this device and is `initialized at the first runtime function` which requires an active context on this device. The context is shared among all the host threads of the application. As part of context creation, the device code is `just-in-time compiled` if necessary and loaded into device memory. This all happens transparently. The primary context created by the CUDA runtime can be accessed from the driver API for interoperability as described in [Interoperability between Runtime and Driver APIs](#).

As of CUDA 12.0, the `cudaInitDevice` and `cudaSetDevice` calls initialize the runtime and the primary `context` associated with the specified device. `The runtime will implicitly use device 0 and self-initialize` as needed to process runtime API requests if they occur before these calls. `This is important when timing runtime function calls and when interpreting the error code from the first call into the runtime`. Prior to CUDA 12.0, `cudaSetDevice` would not initialize the runtime.

`cudaDeviceReset` destroys the `primary context` of the current device. If CUDA runtime APIs are called after the primary context has been destroyed, a new primary context for that device will be created.

Note

The CUDA interfaces use global state that is initialized during host program initiation and destroyed during host program termination. Using any of these interfaces (implicitly or explicitly) during program initiation or termination after main will result in undefined behavior.

As of CUDA 12.0, `cudaSetDevice` explicitly initializes the runtime, if it has not already been initialized, after changing the current device for the host thread. Previous versions of CUDA delayed runtime initialization on the new device until the first runtime call was made after `cudaSetDevice`. Because of this, it is very important to check the return value of `cudaSetDevice` for initialization errors.

The runtime functions from the error handling and version management sections of the reference manual do not initialize the runtime.

2.1.7. Error Checking in CUDA

Every CUDA API returns a value of an enumerated type, `cudaError_t`. In example code these errors are often not checked. In production applications, it is `best practice to always check and manage the return value` of every CUDA API call. When there are no errors, the value returned is `cudaSuccess`. Many applications choose to implement a utility macro such as the one shown below

```
#define CUDA_CHECK(expr_to_check) do { \
    cudaError_t result = expr_to_check; \
    if(result != cudaSuccess) \
    { \
        fprintf(stderr, \
            "CUDA Runtime Error: %s:%i:%d = %s\n", \
            __FILE__, \
            __LINE__, \
            result, \
            cudaGetErrorString(result)); \
    } \
} while(0)
```

This macro `uses the cudaGetErrorString API`, which `returns a human readable string describing the meaning` of a specific `cudaError_t` value. Using the above macro, an application would call CUDA runtime API calls within a `CUDA_CHECK(expression)` macro, as shown below:

```
CUDA_CHECK(cudaMalloc(&devA, vectorLength*sizeof(float)));
CUDA_CHECK(cudaMalloc(&devB, vectorLength*sizeof(float)));
CUDA_CHECK(cudaMalloc(&devC, vectorLength*sizeof(float)));
```

If any of these calls detect an error, it will be printed to `stderr` using this macro. This macro is common for smaller projects, but can be adapted to a logging system or other error handling mechanism in larger applications.

Note

It is important to note that the error state returned from any CUDA API call can also indicate an error from a previously issued asynchronous operation. Section [Asynchronous Error Handling](#) covers this in more detail.

2.1.7.1. Error State

The CUDA runtime maintains a `cudaError_t` state for each host thread. The value defaults to `cudaSuccess` and is overwritten whenever an error occurs. `cudaGetLastError` returns current error state and then resets it to `cudaSuccess`. Alternatively, `cudaPeekLastError` returns error state without resetting it.

Kernel launches using triple chevron notation do not return a `cudaError_t`. It is good practice to check the error state immediately after kernel launches to detect immediate errors in the kernel launch or asynchronous errors prior to the kernel launch. A value of `cudaSuccess` when checking the error state immediately after a kernel launch does not mean the kernel has executed successfully or even started execution. It only verifies that the kernel launch parameters and execution configuration passed to the runtime did not trigger any errors and that the error state is not a previous or asynchronous error before the kernel started.

2.1.7.2. Asynchronous Errors

CUDA kernel launches and many runtime APIs are asynchronous. Asynchronous CUDA runtime APIs will be discussed in detail in [Asynchronous Execution](#). The CUDA error state is set and overwritten whenever an error occurs. This means that errors which occur during the execution of asynchronous operations will only be reported when the error state is examined next. As noted, this may be a call to `cudaGetLastError`, `cudaPeekLastError`, or it could be any CUDA API which returns `cudaError_t`.

When errors are returned by CUDA runtime API functions, the error state is not cleared. This means that error code from an asynchronous error, such as an invalid memory access by a kernel, will be returned by every CUDA runtime API until the error state has been cleared by calling `cudaGetLastError`.

```
vecAdd<<<blocks, threads>>>(devA, devB, devC);
// check error state after kernel launch
CUDA_CHECK(cudaGetLastError());
// wait for kernel execution to complete
// The CUDA_CHECK will report errors that occurred during execution of the kernel
CUDA_CHECK(cudaDeviceSynchronize());
```

Note

The `cudaError_t` value `cudaErrorNotReady`, which may be returned by `cudaStreamQuery` and `cudaEventQuery`, is not considered an error and is not reported by `cudaPeekAtLastError` or `cudaGetLastError`.

2.1.7.3. CUDA_LOG_FILE

Another good way to identify CUDA errors is with the `CUDA_LOG_FILE` environment variable. When this environment variable is set, the CUDA driver will write error messages encountered out to a file whose path is specified in the environment variable. For example, take the following incorrect CUDA code, which attempts to launch a thread block which is larger than the maximum supported by any architecture.

```
__global__ void k()
{ }

int main()
{
    k<<<8192, 4096>>>(); // Invalid block size
    CUDA_CHECK(cudaGetLastError());
    return 0;
}
```

Building and running this, the check after the kernel launch detects and reports the error using the macros illustrated in [Section 2.1.7](#).

```
$ nvcc errorLogIllustration.cu -o errlog
$ ./errlog
CUDA Runtime Error: /home/cuda/intro-cpp/errorLogIllustration.cu:24:1 = invalid argument
```

However, when the application is run with `CUDA_LOG_FILE` set to a text file, that file contains a bit more information about the error.

```
$ env CUDA_LOG_FILE=cudaLog.txt ./errlog
CUDA Runtime Error: /home/cuda/intro-cpp/errorLogIllustration.cu:24:1 = invalid argument
$ cat cudaLog.txt
[12:46:23.854][137216133754880][CUDA][E] One or more of block dimensions of (4096,1,1) exceeds corresponding
[12:46:23.854][137216133754880][CUDA][E] Returning 1 (CUDA_ERROR_INVALID_VALUE) from cuLaunchKernel
```

Setting `CUDA_LOG_FILE` to `stdout` or `stderr` will print to standard out and standard error, respectively. Using the `CUDA_LOG_FILE` environment variable, it is possible to capture and identify CUDA errors, even if the application does not implement proper error checking on CUDA return values. This approach can be extremely powerful for debugging, but the environment variable alone does not allow an application to handle and recover from CUDA errors at runtime. The [error log management](#) feature of CUDA also allows a callback function to be registered with the driver which will be called whenever an error is detected. This can be used to capture and handle errors at runtime, and also to integrate CUDA error logging seamlessly into an application's existing logging system.

[Section 4.8](#) shows more examples of the error log management feature of CUDA. Error log management and `CUDA_LOG_FILE` are available with NVIDIA Driver version r570 and later.

2.1.8. Device and Host Functions

The `__global__` specifier is used to indicate the entry point for a kernel. That is, a function which will be invoked for parallel execution on the GPU. Most often, kernels are launched from the host, however it is possible to launch a kernel from within another kernel using [dynamic parallelism](#).

The specifier `__device__` indicates that a function should be compiled for the GPU and be callable from other `__device__` or `__global__` functions. A function, including class member functions, functors, and lambdas, can be specified as both `__device__` and `__host__` as in the example below.

2.1.9. Variable Specifiers

[CUDA specifiers](#) can be used on static variable declarations to control placement.

- `__device__` specifies that a variable is stored in [Global Memory](#)
- `__constant__` specifies that a variable is stored in [Constant Memory](#)
- `__managed__` specifies that a variable is stored as [Unified Memory](#)
- `__shared__` specifies that a variable is store in [Shared Memory](#)

When a variable is declared with no specifier inside a `__device__` or `__global__` function, it is allocated to registers when possible, and [local memory](#) when necessary. Any variable declared with no specifier outside a `__device__` or `__global__` function will be allocated in system memory.

2.1.9.1. Detecting Device Compilation

When a function is specified with `__host__ __device__`, the compiler is instructed to generate both a GPU and a CPU code for this function. In such functions, it may be desirable to use the preprocessor to specify code only for the GPU or the CPU copy of the function. Checking whether `__CUDA_ARCH__` is defined is the most common way of doing this, as illustrated in the example below.

2.1.10. Thread Block Clusters

From compute capability [9.0 onward](#), the CUDA programming model includes an optional level of hierarchy called [thread block clusters](#) that are made up of thread blocks. [Similar to how threads in a thread block](#) are guaranteed to be co-scheduled on a streaming multiprocessor, [thread blocks in a cluster are also guaranteed to be co-scheduled on a GPU Processing Cluster \(GPC\)](#) in the GPU.

Similar to thread blocks, clusters are also organized into a one-dimension, two-dimension, or three-dimension grid of thread block clusters as illustrated by [Figure 5](#).

The number of thread blocks in a cluster can be user-defined, and a [maximum of 8 thread blocks in a cluster](#) is supported as a portable cluster size in CUDA. Note that on GPU hardware or MIG configurations which are too small to support 8 multiprocessors the maximum cluster size will be reduced accordingly. Identification of these [smaller configurations](#), [as well as of larger configurations](#) supporting a thread block cluster size beyond 8, is architecture-specific and can be [queried](#) using the `cudaOccupancyMaxPotentialClusterSize` API.

All the thread blocks in the cluster are guaranteed to be [co-scheduled to execute simultaneously](#) on a single GPU Processing Cluster (GPC) and [allow](#) thread blocks in the cluster to perform hardware-supported [synchronization](#) using the [cooperative groups](#) API `cluster.sync()`. Cluster group also provides member functions to query cluster group size in terms of number of threads or number of blocks using `num_threads()` and `num_blocks()` API respectively. The rank of a thread or block in the cluster group can be queried using `dim_threads()` and `dim_blocks()` API respectively.

[Thread blocks that belong to a cluster have access to the distributed shared memory](#), which is the [combined shared memory of all thread blocks in the cluster](#). Thread blocks in a cluster have the ability to read, write, and perform atomics to any address in the distributed shared memory. [Distributed Shared Memory](#) gives an example of performing histograms in distributed shared memory.



[Privacy Policy](#) | [Your Privacy Choices](#) | [Terms of Service](#) | [Accessibility](#) | [Corporate Policies](#) | [Product Security](#) | [Contact](#)

Copyright © 2007-2026, NVIDIA Corporation & affiliates. All rights reserved.

Last updated on Mar 04, 2026.